GNO: Graph Neural Operator

Mathematical Formulation and PyTorch Reference Implementation

1 Introduction

The Graph Neural Operator (GNO) is a neural-operator architecture that parameterizes integral operators using graph-based interactions between points in a domain.

The central objective is to learn a mapping

$$\boxed{\mathcal{G} : \mathcal{A} \to \mathcal{U}} \tag{1}$$

between function spaces.

Unlike FNO, which relies on a Fourier representation and therefore naturally operates on regular grids, the graph formulation is designed to represent functions sampled at arbitrary spatial locations.

This makes graph-based operators particularly useful for irregular meshes, point clouds, and nonuniform discretizations. The neural-operator framework introduced graph neural operators as one of its efficient kernel parameterizations. [1]

2 Continuous Operator Formulation

A general neural-operator layer can be expressed as

$$v_{t+1}(x) = \sigma\left(W v_t(x) + \int_D \kappa_\phi\left(x, y, v_t(x), v_t(y)\right) v_t(y)\, dy\right). \tag{2}$$

Here,

- x is the query location,
- y is an integration location,
- v_t is the latent function,
- κ_ϕ is a learned kernel,
- W is a pointwise linear map.

The GNO parameterizes the kernel using a neural network and approximates the integral by summation over neighboring points.

3 Point Cloud Representation

Suppose the domain is represented by points

$$X = \{x_1, \ldots, x_N\}, \qquad x_i \in \mathbb{R}^d. \tag{3}$$

At each point we have a feature

$$v_i = v(x_i) \in \mathbb{R}^C. \tag{4}$$

The complete discretized function is therefore

$$V = \{(x_i, v_i)\}_{i=1}^N. \tag{5}$$

No assumption that the points form a regular Cartesian grid is required.

4 Graph Construction

A graph is constructed over the spatial points:

$$\mathcal{G} = (V, E). \tag{6}$$

For every query point x_i, define a neighborhood

$$\mathcal{N}(i) = \{j : \|x_i - x_j\|_2 \leq r\}, \tag{7}$$

where r is the neighborhood radius.
Alternatively, a k-nearest-neighbor graph can be used:

$$\mathcal{N}(i) = \mathrm{kNN}(x_i). \tag{8}$$

The edge set is

$$E = \{(i, j) : j \in \mathcal{N}(i)\}. \tag{9}$$

5 Kernel Parameterization

The continuous kernel

$$\kappa_\phi\left(x, y, v(x), v(y)\right) \tag{10}$$

is represented by a neural network.
A simple parameterization is

$$\kappa_\phi = \mathrm{MLP}_\phi\left(x - y, v(x), v(y)\right). \tag{11}$$

For a pair of points i, j,

$$e_{ij} = \mathrm{MLP}_\phi\left(x_i - x_j, v_i, v_j\right). \tag{12}$$

The edge feature contains both geometric and feature information.

6 Quadrature Approximation

The continuous integral

$$\int_D \kappa_\phi(x, y) v(y)\, dy \tag{13}$$

is approximated by a weighted sum

$$\sum_{j \in \mathcal{N}(i)} \kappa_\phi(x_i, x_j) v_j \Delta V_j. \tag{14}$$

Here ΔV_j is the quadrature weight associated with point j.

Therefore,

$$\boxed{(\mathcal{K}_\phi V)_i = \sum_{j \in \mathcal{N}(i)} \kappa_\phi(x_i, x_j, v_i, v_j) v_j w_j}. \tag{15}$$

This is the fundamental GNO operation.

7 Message-Passing Interpretation

The same operation can be interpreted as graph message passing.

For every edge (i, j), construct

$$m_{ij} = \kappa_\phi(x_i, x_j, v_i, v_j) v_j. \tag{16}$$

The messages are aggregated at node i:

$$m_i = \sum_{j \in \mathcal{N}(i)} m_{ij} w_j. \tag{17}$$

The node state is then updated according to

$$v_i^{t+1} = \sigma\left(W v_i^t + m_i\right). \tag{18}$$

Thus GNO can be interpreted as a continuous integral operator whose discretization resembles a message-passing graph neural network.

8 Pointwise Linear Transformation

The pointwise component is

$$W v_i = W_t v_i. \tag{19}$$

For channel dimensions

$$W_t \in \mathbb{R}^{C_{\text{out}} \times C_{\text{in}}}. \tag{20}$$

The complete layer becomes

$$\boxed{v_i^{t+1} = \sigma\left(W_t v_i^t + \sum_{j \in \mathcal{N}(i)} \kappa_{\phi_t}(x_i, x_j, v_i^t, v_j^t) v_j^t w_j\right)}. \tag{21}$$

9 Relative-Coordinate Kernel

A common geometry-aware parameterization uses relative coordinates:

$$r_{ij} = x_i - x_j. \tag{22}$$

The kernel becomes

$$\kappa_\phi = \kappa_\phi\left(r_{ij}, v_i, v_j\right). \tag{23}$$

If only relative coordinates are used, the operator is naturally compatible with translations of the coordinate system.

A simple kernel network is

$$\kappa_\phi(r, v_i, v_j) = W_2\sigma\left(W_1[r; v_i; v_j] + b_1\right) + b_2. \tag{24}$$

10 Graph Neural Operator Layer

Define

$$K_\phi(V)_i = \sum_{j\in\mathcal{N}(i)} \kappa_\phi(x_i, x_j, v_i, v_j)v_j w_j. \tag{25}$$

The GNO layer is

$$\boxed{V^{t+1} = \sigma\left(WV^t + K_\phi(V^t)\right)}. \tag{26}$$

Multiple layers give

$$V^L = \mathcal{GNO}_\theta(V^0). \tag{27}$$

11 Input Lifting

Given an input field $a(x)$, the initial feature is

$$v_0(x) = P_{\text{in}}\left(a(x), x\right). \tag{28}$$

At the discrete points,

$$v_i^0 = P_{\text{in}}\left(a_i, x_i\right). \tag{29}$$

The output projection is

$$\hat{u}_i = P_{\text{out}}\left(v_i^L\right). \tag{30}$$

Hence,

$$\boxed{a(\cdot) \to V^0 \to \text{GNO} \to V^L \to \hat{u}(\cdot)}. \tag{31}$$

12 Irregular Geometry

One of the most important differences between GNO and FNO is the representation of the spatial domain.

For FNO, one normally assumes a structured grid:

$$X = \{(x_i, y_j)\}. \tag{32}$$

For GNO, the points may instead be

$$X = \{x_1, \ldots, x_N\}, \tag{33}$$

with arbitrary spatial locations.
Therefore,

$$x_i \not\equiv (i\Delta x, j\Delta y) \tag{34}$$

is allowed.

This makes graph-based operator learning appropriate for irregular domains and unstructured meshes.

13 Complexity

Let each point have approximately K neighbors.

The number of graph edges is approximately

$$|E| \approx NK. \tag{35}$$

If the kernel network has fixed cost per edge, the graph integral has complexity

$$\boxed{\mathcal{O}(NK)}. \tag{36}$$

If $K \ll N$, this is substantially less expensive than a fully connected operator with

$$\mathcal{O}(N^2) \tag{37}$$

pairwise interactions.

However, the neighbor-search and kernel-evaluation costs can become significant for large point clouds.

14 Discretization Perspective

The continuous operator is

$$(\mathcal{K}v)(x) = \int_D \kappa_\phi(x, y, v(x), v(y))v(y)\, dy. \tag{38}$$

The graph approximation is

$$(\mathcal{K}_N v)_i = \sum_{j \in \mathcal{N}(i)} \kappa_\phi(x_i, x_j, v_i, v_j)v_j w_j. \tag{39}$$

As the point cloud becomes sufficiently dense and the quadrature approximation converges,

$$\mathcal{K}_N v \to \mathcal{K}v. \tag{40}$$

This is the key conceptual link between the graph implementation and operator learning.

15 Training Objective

Given training pairs

$$\left\{a^{(n)}, u^{(n)}\right\}_{n=1}^{N}, \tag{41}$$

the standard objective is

$$\mathcal{L} = \frac{1}{N}\sum_{n=1}^{N} \left\|\mathcal{G}_\theta(a^{(n)}) - u^{(n)}\right\|_2^2. \tag{42}$$

A relative error objective is

$$\mathcal{L}_{\text{rel}} = \frac{1}{N}\sum_{n=1}^{N} \frac{\left\|\hat{u}^{(n)} - u^{(n)}\right\|_2}{\left\|u^{(n)}\right\|_2 + \epsilon}. \tag{43}$$

16 PyTorch Implementation

```

import torch
import torch.nn as nn
import torch.nn.functional as F


class KernelNetwork(nn.Module):

    def __init__(
        self,
        coord_dim,
        channels,
        hidden_dim=64
    ):
        super().__init__()

        input_dim = (
            coord_dim
            + 2 * channels
        )

        self.net = nn.Sequential(
            nn.Linear(
                input_dim,
                hidden_dim
            ),
            nn.GELU(),
            nn.Linear(
                hidden_dim,
                hidden_dim
            ),
            nn.GELU(),
            nn.Linear(
                hidden_dim,
                channels
            )
        )

    def forward(
        self,
        relative_coords,
        source_features,
        target_features
    ):

        x = torch.cat(
            [
                relative_coords,
                source_features,
                target_features
            ],
            dim=-1
        )

        return self.net(x)

```

```
57
58  class GNOBlock(nn.Module):
59
60      def __init__(
61          self,
62          channels,
63          coord_dim=2,
64          hidden_dim=64,
65          radius=0.2
66      ):
67          super().__init__()
68
69          self.channels = channels
70          self.radius = radius
71
72          self.kernel = KernelNetwork(
73              coord_dim,
74              channels,
75              hidden_dim
76          )
77
78          self.linear = nn.Linear(
79              channels,
80              channels
81          )
82
83          self.norm = nn.LayerNorm(
84              channels
85          )
86
87      def forward(
88          self,
89          coords,
90          features
91      ):
92
93          B, N, C = features.shape
94
95          # Pairwise relative coordinates
96          relative = (
97              coords.unsqueeze(2)
98              -
99              coords.unsqueeze(1)
100         )
101
102         # Pairwise distances
103         distance = torch.norm(
104             relative,
105             dim=-1
106         )
107
108         # Neighborhood mask
109         mask = (
110             distance
111             <= self.radius
112         )
113
114         # Source features
```

```
        source = (
            features
            .unsqueeze(1)
            .expand(-1, N, -1, -1)
        )

        # Target features
        target = (
            features
            .unsqueeze(2)
            .expand(-1, -1, N, -1)
        )

        # Kernel values
        kernel = self.kernel(
            relative,
            source,
            target
        )

        # Neighbor mask
        kernel = kernel * mask.unsqueeze(-1)

        # Message
        message = (
            kernel
            *
            source
        )

        # Aggregate over neighbors
        message = message.sum(
            dim=2
        )

        # Normalize by number of neighbors
        degree = (
            mask.sum(
                dim=2,
                keepdim=True
            )
            .clamp(min=1)
        )

        message = (
            message
            / degree
        )

        update = (
            self.linear(features)
            +
            message
        )

        return self.norm(
            F.gelu(update)
            + features
```

```
        )


class GNO(nn.Module):

    def __init__(
        self,
        in_channels,
        out_channels,
        coord_dim=2,
        hidden_channels=64,
        depth=4,
        kernel_hidden=64,
        radius=0.2
    ):
        super().__init__()

        self.input_projection = nn.Linear(
            in_channels + coord_dim,
            hidden_channels
        )

        self.blocks = nn.ModuleList([
            GNOBlock(
                channels=hidden_channels,
                coord_dim=coord_dim,
                hidden_dim=kernel_hidden,
                radius=radius
            )
            for _ in range(depth)
        ])

        self.output_projection = nn.Sequential(
            nn.Linear(
                hidden_channels,
                hidden_channels
            ),
            nn.GELU(),
            nn.Linear(
                hidden_channels,
                out_channels
            )
        )

    def forward(
        self,
        coords,
        input_field
    ):

        # coords:
        # [B, N, coord_dim]

        # input_field:
        # [B, N, in_channels]

        x = torch.cat(
            [
```

```
                coords,
                input_field
            ],
            dim=-1
        )

        x = self.input_projection(x)

        for block in self.blocks:

            x = block(
                coords,
                x
            )

        return self.output_projection(x)


if __name__ == "__main__":

    device = (
        "cuda"
        if torch.cuda.is_available()
        else "cpu"
    )

    batch_size = 4
    num_points = 512

    coords = torch.rand(
        batch_size,
        num_points,
        2,
        device=device
    )

    input_field = torch.randn(
        batch_size,
        num_points,
        1,
        device=device
    )

    model = GNO(
        in_channels=1,
        out_channels=1,
        coord_dim=2,
        hidden_channels=64,
        depth=4,
        kernel_hidden=64,
        radius=0.15
    ).to(device)

    output = model(
        coords,
        input_field
    )

```

```
    print(
        "Input:",
        input_field.shape
    )

    print(
        "Output:",
        output.shape
    )
```

17 Training

A standard training procedure is

```

model = GNO(
    in_channels=1,
    out_channels=1,
    coord_dim=2,
    hidden_channels=64,
    depth=4
).to(device)

optimizer = torch.optim.Adam(
    model.parameters(),
    lr=1e-3
)

for epoch in range(100):

    model.train()

    optimizer.zero_grad()

    prediction = model(
        coords,
        input_field
    )

    loss = torch.mean(
        (prediction - target) ** 2
    )

    loss.backward()

    optimizer.step()

    if epoch % 10 == 0:

        print(
            f"Epoch {epoch} | "
            f"Loss = {loss.item():.6e}"
        )
```

18 GNO versus FNO

The two architectures parameterize the nonlocal operator differently.

FNO uses

$$\boxed{\mathcal{K}_\theta v = \mathcal{F}^{-1}\left[R_\theta(k)\mathcal{F}(v)\right]}. \tag{44}$$

GNO uses

$$\boxed{\mathcal{K}_\phi v(x_i) = \sum_{j\in\mathcal{N}(i)} \kappa_\phi(x_i, x_j, v_i, v_j) v_j w_j}. \tag{45}$$

Consequently,

Property	FNO	GNO
Representation	Fourier modes	Graph points
Geometry	Regular grids	Arbitrary point sets
Global interaction	Spectral	Kernel/message passing
Main transform	FFT	Neighborhood aggregation
Irregular meshes	Limited	Natural

19 Summary

The defining GNO operation is

$$\boxed{(\mathcal{K}_\phi v)_i = \sum_{j\in\mathcal{N}(i)} \kappa_\phi(x_i, x_j, v_i, v_j) v_j w_j}. \tag{46}$$

The complete layer is

$$\boxed{v_i^{t+1} = \sigma\left(W_t v_i^t + \sum_{j\in\mathcal{N}(i)} \kappa_{\phi_t}(x_i, x_j, v_i^t, v_j^t) v_j^t w_j\right)}. \tag{47}$$

Thus, GNO can be understood as a learned discretization of a continuous integral operator on a graph.

Its central advantage over grid-dependent architectures is that the operator can act directly on functions represented over irregular spatial locations.

References

[1] N. Kovachki, Z. Li, B. Liu, K. Azizzadenesheli, K. Bhattacharya, A. Stuart, and A. Anandkumar,

Neural Operator: Learning Maps Between Function Spaces,

arXiv:2108.08481, 2021.

https://arxiv.org/abs/2108.08481

[2] Z. Li, N. Kovachki, K. Azizzadenesheli, B. Liu, K. Bhattacharya, A. Stuart, and A. Anandkumar,

Neural Operator: Graph Kernel Network for Partial Differential Equations,

arXiv:2003.03485, 2020.